

Primary sources & further reading

Part 06 — 🧠 AI Agents · 06-ai-agents/papers/PAPERS.md · 06-ai-agents/resources/RESOURCES.md



Papers & Primary Sources — AI Agents

arXiv links are canonical and stable. Read the three starred papers in order before anything else — together they are the entire intellectual history of "give an LLM tools and a loop," and almost everything since is a variation on one of them.

The canon (read in this order)

#	Paper	Year	Why it matters	Link
1	★ ReAct: Synergizing Reasoning and Acting in Language Models — Yao et al.	2022	The pattern this entire module is built around. Read the prompts in Appendix A/B — they are <code>code/react_loop_from_scratch.py</code> , in the original.	arXiv:2210.03629
2	★ Toolformer: Language Models Can Teach Themselves to Use Tools — Schick et al.	2023	Shows a model can learn <i>when</i> to call a tool from self-supervised examples, rather than always from a hand-written prompt. The conceptual root of native tool-calling APIs.	arXiv:2302.04761
3	★ Reflexion: Language Agents with Verbal Reinforcement Learning — Shinn et al.	2023	Self-critique as memory: a verbal reflection on a failed attempt, stored and consulted on the next one. The clearest implementation of procedural memory in this module's taxonomy.	arXiv:2303.11366

Planning & search-based reasoning

Paper	Year	One-line takeaway	Link
Chain-of-Thought Prompting Elicits Reasoning in Large Language Models — Wei et al.	2022	The reasoning half of ReAct, in isolation — read this to see exactly what "acting" adds on top.	arXiv:2201.11903
Tree of Thoughts: Deliberate Problem Solving with Large Language Models — Yao et al.	2023	Branch, evaluate, backtrack — search over reasoning paths instead of committing to one. Expensive; use only where the task is genuinely combinatorial.	arXiv:2305.10601
Plan-and-Solve Prompting — Wang et al.	2023	Explicit upfront decomposition before execution, as a prompting strategy rather than a framework.	arXiv:2305.04091
ReWOO: Decoupling Reasoning from Observations — Xu et al.	2023	Plans the whole tool sequence before executing any of it, cutting redundant model calls versus interleaved ReAct — the efficiency argument against always re-planning per step.	arXiv:2305.18323
Self-Refine: Iterative Refinement with Self-Feedback — Madaan et al.	2023	The single-pass sibling of Reflexion: critique and revise within one episode, no persisted memory required.	arXiv:2303.17651

Tool use & orchestration

Paper	Year	One-line takeaway	Link
MRKL Systems — Karpas et al.	2022	Modular Reasoning, Knowledge and Language: routing between an LLM and external "expert modules" — the architectural ancestor of today's tool router.	arXiv:2205.00445
WebGPT: Browser-assisted Question-Answering — Nakano et al.	2021	An early, concrete agent: search, click, cite — before "agent" was the word anyone used.	arXiv:2112.09332
Gorilla: Large Language Model Connected with Massive APIs — Patil et al.	2023	What happens to tool-selection accuracy as the tool catalog grows into the thousands — direct evidence for notes/02 §2's "too many tools" claim.	arXiv:2305.15334
HuggingGPT — Shen et al.	2023	An LLM as a controller dispatching to specialist models as tools — task decomposition made literal.	arXiv:2303.17580

Memory & long-horizon agents

Paper	Year	One-line takeaway	Link
Generative Agents: Interactive Simulacra of Human Behavior — Park et al.	2023	The "memory stream" architecture — episodic memory with retrieval, reflection, and planning, in a simulated town. The clearest worked example of episodic memory as an implementation, not a metaphor.	arXiv:2304.03442
Voyager: An Open-Ended Embodied Agent with Large Language Models — Wang et al.	2023	A Minecraft agent that writes and saves reusable skills as code — procedural memory that actually accumulates across episodes.	arXiv:2305.16291
A Survey on Large Language Model based Autonomous Agents — Wang et al.	2023	The field-wide map: construction, application, evaluation. Useful after you've read the primary papers above, not before.	arXiv:2308.11432

Official docs (the mechanics, from the source)

Source	What it covers	Link
Anthropic docs	Tool use / function calling: schema format, the request/response round trip, parallel tool calls, error handling	https://docs.claude.com/
Anthropic Cookbook	Runnable tool-use and agent patterns beyond the docs' minimal examples	https://github.com/anthropics/anthropic-cookbook
OpenAI docs	Function calling: the same mechanics from a second vendor, useful for spotting what's universal vs. provider-specific	https://platform.openai.com/docs/guides/function-calling

How to read an agent paper (30 minutes, 3 passes)

1. **Pass 1 (5 min)**: abstract, the main diagram (almost always a loop or a tree), and the headline benchmark number. What's the claimed capability, and on what task family?
2. **Pass 2 (15 min)**: find the actual prompt in the appendix. Agent papers live or die on prompt engineering that the abstract never mentions — read the literal Thought/Action text before believing the architecture diagram.
3. **Pass 3 (10 min)**: the failure analysis, if there is one (not all agent papers have one — treat that absence as a yellow flag). Which failure modes from [notes/02 §3](#) does this paper's method actually address, and which does it just not encounter in its benchmark?

Agent benchmarks are notoriously easy to overfit to (a fixed tool set, a narrow task distribution) — the mechanism in a paper generalizes far more reliably than its reported success rate does.

Resources — AI Agents

Courses & lectures (free)

Resource	Why	Link
LLM Agents MOOC (Berkeley)	The most rigorous free course on this exact material — reasoning, planning, tool use, memory, evaluation, taught by the people who wrote several of the canon papers	https://llmagents-learning.org/f24
DeepLearning.AI — Functions, Tools and Agents with LangChain	Short, hands-on: the tool-calling round trip and the ReAct loop, built incrementally	https://www.deeplearning.ai/short-courses/functions-tools-agents-langchain/
DeepLearning.AI — AI Agents in LangGraph	The runtime side of this module — pairs directly with <code>../.. /16-ai-tech-stack/tracks/langgraph/</code>	https://www.deeplearning.ai/short-courses/ai-agents-in-langgraph/
Prompting Guide — LLM Agents	A concise, well-maintained reference on ReAct, planning and agent architectures, with links back to the primary papers	https://www.promptingguide.ai/research/llm-agents

Repositories worth cloning

Repo	What's inside
https://github.com/ysymyth/ReAct	The original ReAct paper's code and prompts — read the actual few-shot exemplars
https://github.com/noahshinn/reflexion	Reference implementation of Reflexion's verbal self-critique loop
https://github.com/princeton-nlp/tree-of-thought-llm	Tree of Thoughts reference implementation, including the scoring/pruning logic
https://github.com/ShishirPatil/gorilla	Large-scale API/tool-calling benchmark and models — the "too many tools" evidence in code form
https://github.com/anthropics/anthropic-cookbook	Runnable tool-use and agent recipes against the Claude API
https://github.com/langchain-ai/langchain	The tool/model abstraction layer most agent frameworks build on — see <code>../.. /16-ai-tech-stack/tracks/langchain/</code>
https://github.com/langchain-ai/langgraph	The durable, resumable agent-loop runtime — see <code>../.. /16-ai-tech-stack/tracks/langgraph/</code>

Cheat sheets

- Anthropic docs (tool use, agent SDK, agent design patterns): <https://docs.claude.com/>
- OpenAI function calling guide: <https://platform.openai.com/docs/guides/function-calling>
- Also see `../.. /_shared/cheatsheets/`

Where the theory meets production (this repo)

Concern	Where
Durable, resumable agent runtime; cycles, checkpoints, human-in-the-loop	<code>../../../../16-ai-tech-stack/tracks/langgraph/</code>
The model/tool abstraction layer, concrete implementation	<code>../../../../16-ai-tech-stack/tracks/langchain/</code>
A real agent harness applying these exact recovery/permission patterns	<code>../../../../16-ai-tech-stack/tracks/claude-code/</code>
Multi-agent topologies, hand-offs, supervisors	Module 07 — Agentic AI Systems [07-agentic-ai]
The lethal trifecta, prompt injection, tool-access abuse	Module 13 — AI Security [13-ai-security]
Trajectory grading, agent eval harnesses	Module 15 — AI Evaluation [15-ai-evals]
Standardizing tool/data access across agents	Module 09 — MCP [09-mcp]

Communities

- r/LocalLLaMA and r/LangChain — active discussion of what actually breaks in agent deployments, not just demos
- LangChain / LangGraph Discord — the people building the runtime this module's theory feeds into
- Hugging Face forums (`agents` tag) — <https://discuss.huggingface.co/>